1)**Initialize a new repository using Git. Create a README.md file and commit it to the repository with an appropriate commit message.**

Step 1: Create a project folder

mkdir GitPractice

cd GitPractice

Step 2: Initialize a new Git repository

git init

Output:

Initialized empty Git repository

Step 3: Create a README.md file

echo # My Git Project > README.md

This creates a README.md file containing the project heading.

Step 4: Check the repository status

git status

The output will show README.md as an untracked file.

Step 5: Add the file to the staging area

git add README.md

Step 6: Commit the file

git commit -m "Add README file"

Expected output:

[master (root-commit) abc1234] Add README file

1 file changed, 1 insertion(+)

create mode 100644 README.md

Note: The commit hash and branch name may differ.

Step 7: Verify the commit

git log --oneline

Output:

abc1234 Add README file

2) Develop a simple "Hello World" web application using either Python (Flask) or Node.js. Explain the code structure and run the application locally

Step 1: Create a project folder

Open Command Prompt (CMD) and type:

mkdir HelloWorld

cd HelloWorld

npm init -y

This creates a project folder and a package.json file.

Step 2: Install Express.js

npm install express

Express is a Node.js framework used to create web applications.

Step 3: Create app.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
   res.send('Hello World!');
});

app.listen(3000, () => {
   console.log('Server running at http://localhost:3000');
});
```

Step 4: Understand the code

Code	Purpose
require('express')	Imports Express
express()	Creates an application
app.get('/')	Handles requests to the home page
res.send()	Sends the Hello World message
app.listen(3000)	Starts the server on port 3000

Step 5: Run the application

In CMD, execute:

node app.js

Expected output in CMD:

Server running at http://localhost:3000

Step 6: Open the application in a browser

Open Chrome and enter:

http://localhost:3000

Hello World!

Expected browser output

Step 7: Code structure

HelloWorld/

│

├── node_modules/

├── app.js

├── package.json

└── package-lock.json

- app.js — contains the application code.
- package.json — stores project details and dependencies.
- package-lock.json — records exact dependency versions.
- node_modules/ — contains installed packages.

Result

Successfully developed and executed a simple Hello World web application using Node.js and Express.js. The application runs locally on port 3000 and displays Hello World! in the browser.

3) Perform steps required to execute your web application on a local machine. Include commands and expected output in the browser.

Step 1: Create a project folder

Open CMD and execute:

mkdir MyWebApp

cd MyWebApp

npm init -y

Step 2: Install Express

npm install express

Step 3: Create app.js

Create a file named app.js inside the MyWebApp folder and write this code:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
   res.send('Welcome to My Web Application!');
});

app.listen(3000, () => {
   console.log('Server running at http://localhost:3000');
});
```

Step 4: Run the web application

Execute this command in CMD:

node app.js

Expected output in CMD:

Server running at http://localhost:3000

Step 5: Open the application in a browser

Open Google Chrome and enter the following URL:

http://localhost:3000

Welcome to My Web Application!

Expected browser output

Step 6: Project structure

MyWebApp/

├── node_modules/

├── app.js

├── package.json

└── package-lock.json

Result

Successfully executed the web application on the local machine using Node.js and Express.js. The server runs on port 3000, and the browser displays "Welcome to My Web Application!" when accessed at http://localhost:3000.

2

Q.1 Demonstrate a Dockerfile for your application. Explain each instruction used in the Dockerfile and its purpose in containerization.

Aim: To create a Dockerfile for a Node.js web application and run it inside a Docker container.

1. Create a Dockerfile

Create a file named Dockerfile (without any file extension) in the project folder alongside app.js and package.json.

Project structure:

MyWebApp/

├— app.js

├— package.json

├— package-lock.json

└— Dockerfile

2. Dockerfile code

```
# Use the Node.js base image

FROM node:20


# Set the working directory

WORKDIR /app


# Copy package files

COPY package*.json ./
```

```
# Install dependencies
RUN npm install

# Copy application files
COPY . .

# Expose application port
EXPOSE 3000

# Start the application
CMD ["node", "app.js"]
```

3. Explanation of each instruction

Instruction	Purpose
FROM node:20	Specifies the base image containing Node.js.
WORKDIR /app	Sets the working directory inside the container.
COPY package*.json ./	Copies package files into the container.
RUN npm install	Installs the required dependencies.
COPY . .	Copies the application source code into the container.
EXPOSE 3000	Documents the port used by the application.
CMD ["node", "app.js"]	Runs the application when the container starts.

4. Build the Docker image

Open CMD in the project folder and execute:

```
docker build -t mywebapp .
```

Expected output:

```
[+] Building ... FINISHED
Successfully tagged mywebapp:latest
```

The exact output may vary depending on the Docker version.

5. Run the Docker container

```
docker run -d -p 3000:3000 --name mycontainer mywebapp
```

- -d — runs the container in the background.

- -p 3000:3000 — maps port 3000 on the host to port 3000 in the container.
- --name mycontainer — assigns a name to the container.
- mywebapp — specifies the image to run.

6. Check the running container

docker ps

Expected output:

CONTAINER ID IMAGE STATUS PORTS

abc123 mywebapp Up 0.0.0.0:3000->3000/tcp

The container ID and other displayed values will differ.

7. View the application in the browser

Open Google Chrome and enter:

http://localhost:3000

Expected browser output:

localhost:3000

Welcome to My Web Application

My web application is running successfully!

Result

The Dockerfile was created successfully. A Docker image was built, a container was started, and the Node.js web application was accessed through the browser.

2)Perform the process of building a Docker image from your application. Include the command used and describe the output generated

Practical: Building a Docker Image from an Application

Aim: To build a Docker image for a simple Node.js application using a Dockerfile.

Step 1: Create a Dockerfile

Create a file named Dockerfile in your application folder and write:

FROM node:20

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "app.js"]

Step 2: Build the Docker image

Open CMD in your application folder, for example D:\Practice-code\question2, and execute:

docker build -t question2 .

Explanation of the command:

- docker build — builds a Docker image.
- -t question2 — assigns the image the name question2.
- . — specifies the current directory as the build context.

Step 3: Expected output

Docker displays build steps similar to the following:

[+] Building ... FINISHED

=> [internal] load build definition from Dockerfile

=> [internal] load metadata for docker.io/library/node:20

=> [1/5] FROM docker.io/library/node:20

=> [2/5] WORKDIR /app

=> [3/5] COPY package*.json ./

=> [4/5] RUN npm install

=> [5/5] COPY . .

=> exporting to image

=> naming to docker.io/library/question2:latest

Note: The exact output and step numbers may vary.

Step 4: Verify the Docker image

Execute:

docker images

Expected output (illustrative):

REPOSITORY TAG IMAGE ID CREATED SIZE

question2 latest abc123def456 moments ago 1.2GB

The image ID, creation time, and size will vary.

Result

Successfully built a Docker image named question2 from the Node.js application using the Dockerfile. The image can now be used to create and run a Docker container.

3) Run a Docker container from the created image. Explain port mapping and how to access the application in a web browser.

step 1: Create the project files

Open CMD and run these commands:

mkdir question2

cd question2

notepad index.html

Paste this code into index.html and save it.

```
<!DOCTYPE html>
<html>
<head>
  <title>My Docker Application</title>
</head>
<body>
  <h1>Welcome to My Docker Application!</h1>
  <p>My web application is running successfully inside a Docker container.</p>
</body>
</html>
```

Step 2: Create the Dockerfile

In CMD, run:

notepad Dockerfile

Paste this code and save the file. Make sure the filename is exactly Dockerfile, without a .txt extension.

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Explanation:

- FROM nginx:alpine — Uses a lightweight Nginx web server image.
- COPY index.html /usr/share/nginx/html/index.html — Copies your HTML file into the web server's default directory.
- EXPOSE 80 — Documents that the application uses container port 80.

Step 3: Build the Docker image

Run this command inside the question2 folder:

docker build -t myimage .

Expected output includes a successful build message and the image name myimage:latest.

Check the image:

docker images

Step 4: Run the Docker container

docker run -d -p 3001:80 --name mycontainer myimage

Explanation:

- -d — Runs the container in the background.
- -p 3001:80 — Maps host port 3001 to container port 80.
- --name mycontainer — Gives the container a name.
- myimage — Uses the image created in Step 3.

Check whether the container is running:

docker ps

Step 5: Open the application in your browser

Open Chrome and enter:

http://localhost:3001

Welcome to My Docker Application!

My web application is running successfully inside a Docker container.

Step 6: Stop the container (optional)

docker stop mycontainer

To remove the stopped container:

docker rm mycontainer

Remember the practical sequence: Create index.html → Create Dockerfile → Build image → Run container → Open browser.

1) Create a new branch named feature-app in your local Git repository and switch to that branch. Verify the active branch using an appropriate command.

Step 1: Open CMD in Your Project Folder

cd path\to\your\project

Step 2: Create and Switch to the Branch

git checkout -b feature-app

Output:

Switched to a new branch 'feature-app'

This command creates the feature-app branch and switches to it.

Step 3: Verify the Active Branch

git branch

Output:

* feature-app

  Main

2) Create a remote repository on GitHub and push your local project (including all commits) to the remote repository. Show the commands used.

Step 1: Create a Repository on GitHub

1. Open GitHub.
2. Log in to your account.
3. Click the + icon and select New repository.
4. Enter a repository name, for example, my-project.
5. Select Public or Private.
6. Click Create repository.

Keep the repository empty when creating it; do not initialize it with a README, .gitignore, or license if your local project already has commits.

Step 2: Open CMD in Your Project Folder

cd path\to\your\project

For example:

cd D:\Practice-code\question2

Step 3: Check Git Status

git status

This command displays the current branch and working tree status.

Step 4: Add the GitHub Remote Repository

Replace YOUR-USERNAME with your GitHub username.

git remote add origin https://github.com/YOUR-USERNAME/my-project.git

Verify the remote URL:

git remote -v

Expected output:

origin https://github.com/YOUR-USERNAME/my-project.git (fetch)

origin https://github.com/YOUR-USERNAME/my-project.git (push)

Step 5: Push All Local Commits to GitHub

If your active branch is feature-app, execute:

git push -u origin feature-app

Expected output (example):

Enumerating objects: 10, done.

Counting objects: 100% (10/10), done.

Writing objects: 100% (10/10), done.

To https://github.com/YOUR-USERNAME/my-project.git

 * [new branch]   feature-app -> feature-app

branch 'feature-app' set up to track 'origin/feature-app'.

The exact output will depend on your project.

The git push -u origin feature-app command uploads the branch and its committed history to GitHub. The -u option sets the upstream branch for future pushes.

Step 6: Verify the Push

git branch -a

Expected output:

* feature-app

remotes/origin/feature-app

3) Run the Docker container for your application using the built image. Map the container port to the host system and access the application through a web browser

Step 1: Check the Docker Image

Open CMD in your project folder and run:

docker images

Expected output:

REPOSITORY TAG IMAGE ID CREATED SIZE

my-app latest abc123456789 2 minutes ago 150MB

Here, my-app is an example image name. Use the actual name of your image.

Step 2: Run the Docker Container

docker run -d -p 3001:80 --name my-container my-app

Replace my-app with your built image name.

Explanation:

- docker run — creates and starts a container.
- -d — runs the container in the background.
- -p 3001:80 — maps host port 3001 to container port 80.
- --name my-container — assigns a name to the container.
- my-app — specifies the Docker image.

Port mapping

Host

3001

Container

80

Requests to your computer's port 3001 are forwarded to port 80 inside the container.

Note: This example assumes your application listens on port 80 inside the container. If it listens on port 3000, use -p 3001:3000 instead.

Step 3: Verify the Running Container

docker ps

Expected output:

CONTAINER ID IMAGE PORTS NAMES

abc123 my-app 0.0.0.0:3001->80/tcp my-container

Step 4: Access the Application in a Web Browser

Open Chrome or another browser and enter:

http://localhost:3001

Your application should appear in the browser.

Step 5: Stop the Container (Optional)

docker stop my-container

Result

Successfully ran the Docker container from the built image, mapped the container port to host port 3001, and accessed the application through a web browser.

----------4----------

1) Create a new branch (e.g., feature-update), modify your application (change output message), commit the changes, and push the branch to GitHub.

Step 1: Open your project folder

cd C:\Practice-code\question2

Replace the path if your project is in a different folder.

Step 2: Create a new branch and switch to it

git checkout -b feature-update

Output:

Switched to a new branch 'feature-update'

Verify the active branch:

git branch

Output:

* feature-update

main

Step 3: Modify your application

If you are using a simple HTML application, open index.html and change the output message.

```
<!DOCTYPE html>
<html>
<head>
   <title>My Application</title>
</head>
<body>
   <h1>Hello, Welcome to My Updated Application!</h1>
</body>
</html>
```

Save the file.

Step 4: Check the changes

```
git status
```

Expected output:

```
On branch feature-update
Changes not staged for commit:
  modified: index.html
```

Step 5: Stage and commit the changes

```
git add .
git commit -m "Update application output message"
```

Expected output:

```
1 file changed, 1 insertion(+), 1 deletion(-)
```

The exact output may differ depending on your changes.

Step 6: Push the branch to GitHub

If your GitHub remote repository is already configured, run:

```
git push -u origin feature-update
```

Expected output will include something similar to:

```
To https://github.com/username/repository.git
```

* [new branch]   feature-update -> feature-update

branch 'feature-update' set up to track 'origin/feature-update'.

If the remote is not configured, add your GitHub repository URL first:

git remote add origin https://github.com/USERNAME/REPOSITORY.git

git push -u origin feature-update

Replace USERNAME and REPOSITORY with your actual GitHub details.

Step 7: Verify the result

git branch

git log -1 --oneline

git branch -r

Expected results:

- Active branch: feature-update
- Latest commit: Update application output message
- Remote branch: origin/feature-update

Open your GitHub repository in a browser and select the feature-update branch to verify the updated code.

Result: A new branch was created, the application output message was modified, the changes were committed, and the branch was pushed to GitHub.

2) Create a Dockerfile for your application. Build the Docker image and display the list of available images on your system.

Step 1: Create a Dockerfile

Open your project folder:

cd C:\Practice-code\question2

notepad Dockerfile

Paste this code into Notepad and save the file as Dockerfile (without any extension).

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

Explanation of each instruction:

- FROM nginx:alpine — Uses the lightweight Nginx web server image.
- COPY index.html /usr/share/nginx/html/index.html — Copies your HTML application into the web server directory.

- EXPOSE 80 — Documents the port used by the web server inside the container.

Step 2: Build the Docker image

Run this command from the project folder:

docker build -t my-web-app .

Here, -t my-web-app assigns a name to the image, and . specifies the current directory as the build context.

Expected output (simplified):

[+] Building ... FINISHED

=> [internal] load build definition from Dockerfile

=> [internal] load metadata for docker.io/library/nginx:alpine

=> [internal] COPY index.html /usr/share/nginx/html/index.html

=> exporting to image

=> naming to docker.io/library/my-web-app

The exact output may differ depending on your Docker version and whether the base image is already downloaded.

Step 3: Display all available Docker images

Execute:

docker images

Expected output:

REPOSITORY TAG IMAGE ID CREATED SIZE

my-web-app latest abc123def456 A few seconds ago 50MB

nginx alpine def456abc123 A few days ago 40MB

These image IDs, sizes, and dates are examples. Your actual output will be different.

Step 4: Verify your image

To display only your newly created image, run:

docker images my-web-app

You should see the my-web-app image with the latest tag.

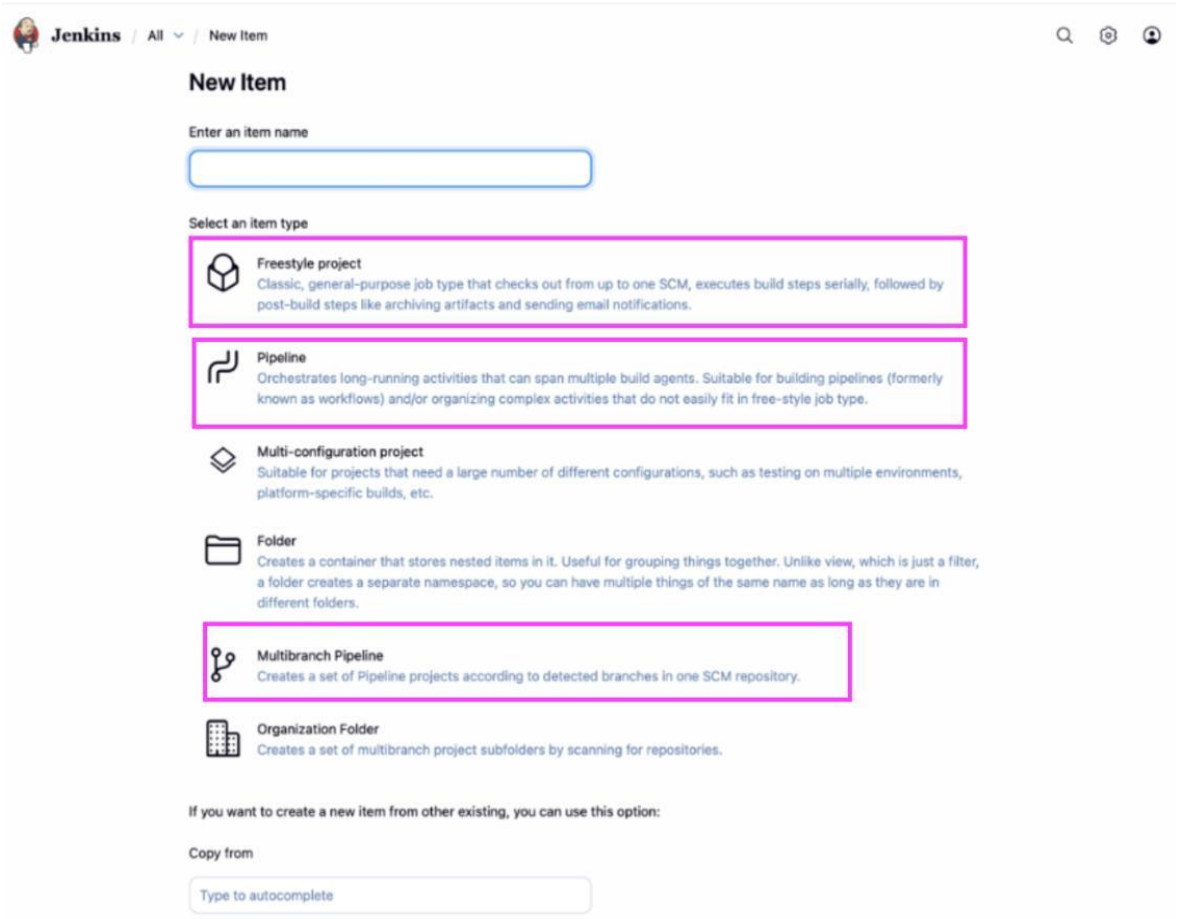
Result: A Dockerfile was created, the Docker image was built successfully, and the list of available Docker images was displayed.

3) Create a new Pipeline project in Jenkins. Configure the basic settings required to run a pipeline.

Step 1: Open Jenkins

1. Start Jenkins on your computer.
2. Open your web browser and visit http://localhost:8080.
3. Log in using your Jenkins username and password.

Step 2: Create a new Pipeline project



1. Click New Item

On the Jenkins dashboard, click New Item.

2. Enter project details

- Item name: MyFirstPipeline
- Select Pipeline
- Click OK

Step 3: Configure the basic settings

On the configuration page:

1. General: Enter a description such as My first Jenkins pipeline project.
2. Build Triggers: Leave the options unchecked for a manual run.
3. Pipeline: Select Pipeline script in the Definition dropdown.
4. Enter the following script in the Script box.

```
pipeline {
    agent any

    stages {
```

```
        stage('Build') {
            steps {
                echo 'Building the application'
            }
        }

        stage('Test') {
            steps {
                echo 'Testing the application'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying the application'
            }
        }
    }
}
```

5. Click Save.

Step 4: Run the pipeline

1. Open the MyFirstPipeline project.
2. Click Build Now.
3. Open the build number under Build History.
4. Click Console Output to see the execution logs.

Expected output:

```
[Pipeline] Start of Pipeline
[Pipeline] stage (Build)
Building the application
[Pipeline] stage (Test)
Testing the application
[Pipeline] stage (Deploy)
Deploying the application
[Pipeline] End of Pipeline
```

Finished: SUCCESS

The exact console formatting may vary.

Result

A new Pipeline project named MyFirstPipeline was created in Jenkins, its basic settings were configured, and the pipeline was executed successfully.

1) Merge the feature-app branch into the main branch. Verify that the changes are successfully merged and push the updated main branch to GitHub.

Step 1: Open the project folder

cd C:\Practice-code\question2

Step 2: Check the available branches

git branch

Expected output:

feature-app

* main

If feature-app exists, continue. If it does not appear, check your branch names before proceeding.

Step 3: Switch to the main branch

git checkout main

Expected output:

Switched to branch 'main'

Step 4: Merge the feature-app branch

git merge feature-app

Expected output (example):

Updating abc1234..def5678

Fast-forward

index.html | 2 +-

1 file changed, 1 insertion(+), 1 deletion(-)

The output may differ depending on your commits. If Git reports a merge conflict, resolve it before continuing.

Step 5: Verify the merged changes

Check the commit history:

git log --oneline --graph --all

Check the working tree:

git status

Expected output:

On branch main

nothing to commit, working tree clean

To verify that the changes from feature-app are included in main, run:

git merge-base --is-ancestor feature-app main

echo %ERRORLEVEL%

Expected output:

0

A result of 0 means the feature branch's tip is included in main.

Step 6: Push the updated main branch to GitHub

git push origin main

Expected output (example):

To https://github.com/username/repository.git

  abc1234..def5678 main -> main

Result

The feature-app branch was merged into main, the merged changes were verified using Git commands, and the updated main branch was pushed to GitHub.

2) Configure your Jenkins pipeline to pull the project source code from your GitHub repository. Demonstrate the configuration steps.

Step 1: Open Jenkins

1. Open your browser and visit http://localhost:8080.
2. Log in to Jenkins.
3. Click New Item.
4. Enter the project name GitHub-Pipeline.
5. Select Pipeline and click OK.

Step 2: Copy your GitHub repository URL

Open your project repository on GitHub, click the green Code button, and copy the HTTPS URL.

Example:

https://github.com/USERNAME/REPOSITORY.git

Replace this example with your actual repository URL.

Step 3: Configure the pipeline

Scroll down to the Pipeline section on the Jenkins configuration page.

- Definition: Select Pipeline script.
- In the Script box, enter the following code:

```
pipeline {
   agent any


   stages {
      stage('Checkout') {
         steps {
            git branch: 'main',
               url: 'https://github.com/USERNAME/REPOSITORY.git'
         }
      }


      stage('Verify Source Code') {
         steps {
            bat 'dir'
         }
      }
   }
}
```

Replace USERNAME/REPOSITORY.git with your actual GitHub repository URL.

The git step downloads the source code from the main branch into the Jenkins workspace. The bat 'dir' command lists the downloaded files on a Windows Jenkins agent.

Step 4: Save and run the pipeline

1. Click Save.
2. Open the GitHub-Pipeline project.
3. Click Build Now.
4. Open the build number under Build History.
5. Click Console Output.

Step 5: Verify the output

Expected output (simplified):

[Pipeline] Start of Pipeline

[Pipeline] stage

[Pipeline] { (Checkout)

Cloning the remote Git repository

Checking out Revision ...

[Pipeline] }

[Pipeline] stage

[Pipeline] { (Verify Source Code)

Directory of ...\workspace\GitHub-Pipeline

index.html

Dockerfile

...

[Pipeline] }

[Pipeline] End of Pipeline

Finished: SUCCESS

The filenames and console messages depend on your repository and Jenkins version.

Note: For a private GitHub repository, configure credentials in Manage Jenkins → Credentials and use the appropriate credential ID in the pipeline. The Jenkins agent must also have Git installed.

Result

A Jenkins Pipeline project was configured to pull source code from the GitHub repository, and the checkout was verified through the Jenkins Console Output.

Q.3 Create and execute a Jenkins pipeline with stages to:

1. **Clone the repository**
2. **Build the Docker image**
3. **Run the Docker container**

Verify that the application runs successfully after pipeline execution.

We will use your existing web application with index.html and Dockerfile, GitHub, Jenkins, and Docker on Windows.

Step 1: Check your project files

Your GitHub repository should contain these files:

question2/

├── index.html

└── Dockerfile

Example Dockerfile:

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

Commit and push these files to your GitHub main branch before running the pipeline.

Step 2: Open Jenkins and create a Pipeline

1. Open http://localhost:8080.
2. Click New Item.
3. Enter the name Docker-Pipeline.
4. Select Pipeline, then click OK.
5. Scroll down to the Pipeline section.
6. Set Definition to Pipeline script.

Paste the following script into the Script box.

```
pipeline {
  agent any


  stages {
    stage('Clone Repository') {
      steps {
        git branch: 'main',
```

```
                url: 'https://github.com/USERNAME/REPOSITORY.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                bat 'docker build -t my-web-app .'
            }
        }

        stage('Run Docker Container') {
            steps {
                bat 'docker rm -f my-web-container || exit /b 0'
                bat 'docker run -d --name my-web-container -p 8081:80 my-web-app'
            }
        }

        stage('Verify Application') {
            steps {
                bat 'curl.exe -f http://localhost:8081'
            }
        }
    }
}
```

Replace USERNAME/REPOSITORY.git with your actual GitHub repository URL.

Step 3: Execute the pipeline

1. Click Save.
2. Open Docker-Pipeline.
3. Click Build Now.
4. Click the build number in Build History.
5. Select Console Output to see the execution logs.

Before running, ensure Docker Desktop is running and the Jenkins agent can execute Docker commands. This example assumes Jenkins runs on Windows and can access Docker on the same machine.

Step 4: Verify the Docker container

Open CMD and run:

docker images

Expected: the my-web-app image appears in the list.

Check the running container:

docker ps

Expected output will contain a row similar to:

CONTAINER ID IMAGE STATUS PORTS

abc123 my-web-app Up ... 0.0.0.0:8081->80/tcp

Step 5: Verify the application in a browser

Open:

http://localhost:8081

You should see your web application and the updated output message from index.html.

The curl.exe verification stage also checks that the web server returns a successful HTTP response. If it fails, Jenkins marks the pipeline as failed.

Expected pipeline stages

1. Clone Repository

Downloads source code from GitHub.

2. Build Docker Image

Creates the my-web-app image.

3. Run Docker Container

Starts the application on host port 8081.

4. Verify Application

Checks the HTTP response from the application.

Result: A Jenkins pipeline was created to clone the GitHub repository, build a Docker image, run the container, and verify that the web application is accessible. When all stages succeed, the Jenkins build status displays SUCCESS.